

UNISAL

Centro Universitário Salesiano de São Paulo

SISTEMA DE CONTROLE DE ESTOQUE DE PRODUTOS PERECÍVEIS

Documento Arquitetural Completo

Engenharia de Software

Integrantes

Ana Beatriz de Oliveira Elias

Elisa da Silva Ortiz de Godoy

Larissa dos Santos Oliveira

Vinicius Gama Bittencourt

Lorena - SP

2026

1 Evolução do Diagrama de Classes

1.1 Diagrama de Classes – Versão Original

A versão original do diagrama de classes foi construída diretamente a partir dos requisitos funcionais levantados na Parte 1 do projeto. Ela representa a estrutura inicial do Sistema de Controle de Estoque de Produtos Perecíveis, contemplando as entidades principais sem a aplicação formal de padrões de projeto.

Classes identificadas na versão original

- **Pessoa:** representa os dados básicos dos indivíduos cadastrados no sistema, contendo atributos como nome, CPF, número de celular, e-mail, data de nascimento e residência. Possui operações de cadastro, consulta, edição, remoção e validação de CPF.

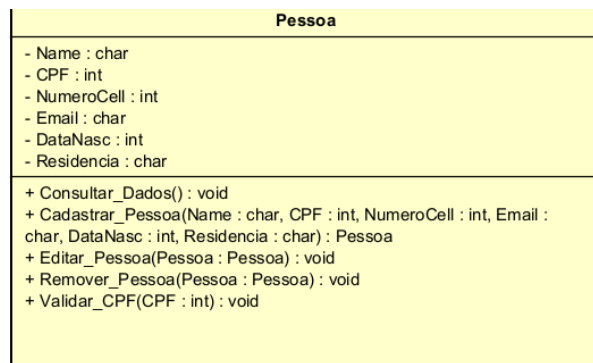


Figura 1: Diagrama: Classe Pessoa

- **Funcionário:** especialização da classe **Pessoa**, responsável pelo gerenciamento dos usuários internos do sistema. Possui atributos relacionados ao cargo, salário, situação e senha, além de operações de cadastro, edição e autenticação.

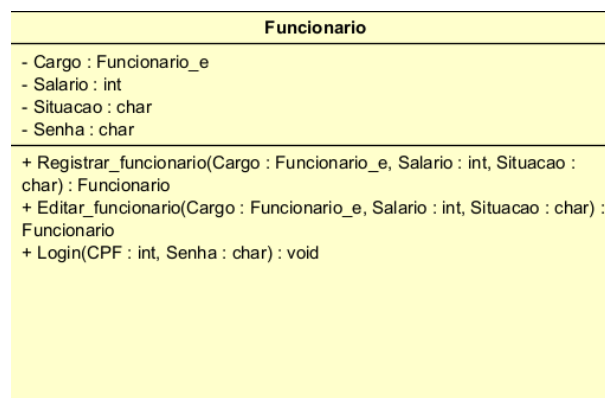


Figura 2: Diagrama: Classe Funcionário

- **Funcionario_e:** enumeração responsável pela definição dos cargos existentes no sistema, sendo composta pelos valores *Gerente* e *Estoquista*.

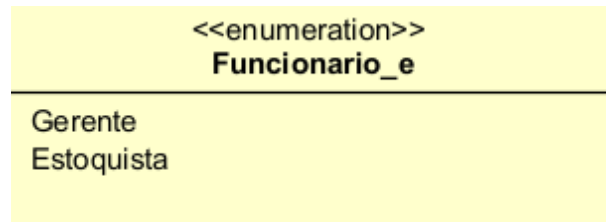


Figura 3: Diagrama: Classe Funcionário_e

- **Fornecedor:** representa as empresas responsáveis pelo abastecimento do estoque, armazenando informações como nome, CNPJ, empresa e logradouro. Também possui operações de cadastro, consulta, atualização e configuração de rotas.

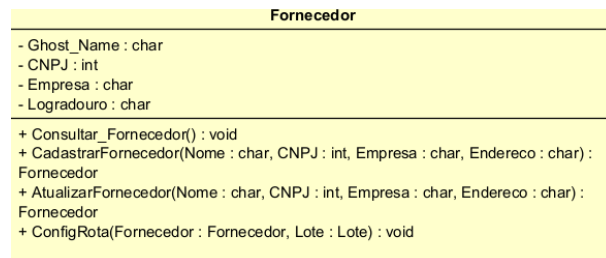


Figura 4: Diagrama: Classe Fornecedor

- **Produto:** representa os itens armazenados no estoque, contendo atributos como código e código de barras. Possui operações de cadastro, consulta e atualização.

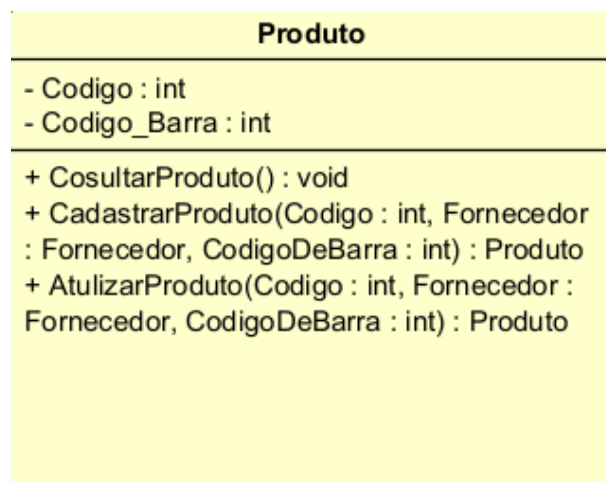


Figura 5: Diagrama: Classe Produto

- **Lote:** representa os lotes dos produtos armazenados, contendo informações referentes ao código, produto associado, validade, quantidade, data de recebimento e data de fabricação. Além disso, concentra operações de cadastro, consulta, controle de validade, alerta de quantidade, registro de perdas, criação de relatórios e visualização do prazo de giro dos produtos.

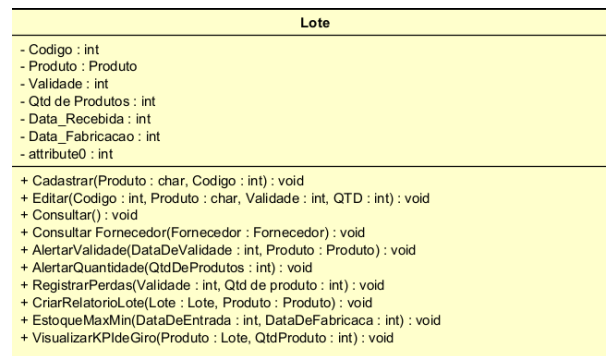


Figura 6: Diagrama: Classe Lote

Problemas identificados na versão original

- A classe **Lote** acumulava diversas responsabilidades relacionadas ao controle do estoque, monitoramento da validade, emissão de alertas, registro de perdas e geração de relatórios, caracterizando uma violação do Princípio da Responsabilidade Única (SRP).
- Não existia uma classe central responsável pelo gerenciamento das operações do estoque, dificultando a organização e a manutenção do sistema.
- As regras de controle dos produtos estavam fortemente acopladas às classes do domínio, reduzindo a flexibilidade da arquitetura.
- Os módulos externos precisavam acessar diretamente diversas classes para executar operações do sistema, aumentando o acoplamento entre os componentes.
- Não havia uma abstração para diferentes estratégias de gerenciamento e reposição dos produtos, tornando a lógica rígida e dificultando futuras extensões.
- A estrutura geral do sistema apresentava baixa modularidade, reduzindo a reutilização de código e dificultando a evolução da arquitetura.

1.2 Diagrama de Classes – Versão Evoluída

A versão evoluída incorpora os padrões de projeto do catálogo Gang of Four (GoF), com o objetivo de corrigir os problemas identificados e aumentar a manutenibilidade, extensibilidade e coesão do sistema.

Classes e interfaces adicionadas

- **FuncionarioFactory** (*<interface><FactoryMethod>*): define a interface para o método de fabricação de funcionários, estabelecendo o contrato para a aplicação do padrão Factory Method.
- **GerenteFactory** e **EstoquistaFactory**: classes de fábricas concretas que implementam a interface **FuncionarioFactory**. Elas encapsulam a lógica de instanciação

isolada, sendo responsáveis por criar e retornar os objetos específicos **Gerente** e **Estoquista**, respectivamente.

- **Gerente** e **Estoquista** (subclasses): especializações físicas criadas a partir da classe **Funcionario**. Substituem o uso antigo de enumeradores por herança pura, permitindo que cada tipo de funcionário possua comportamentos e regras de negócio isoladas, explorando o polimorfismo.

1.3 Principais Alterações Estruturais

A Tabela 1 apresenta as principais diferenças entre a versão original e a versão evoluída do sistema após a aplicação dos padrões de projeto.

Tabela 1: Comparativo entre a Versão Original e a Versão Evoluída

Aspecto	Versão Original	Versão Evoluída
Padrão de Criação de Objetos	Instanciação direta e acoplada no código cliente ou condicionada por flags de tipo.	Descentralização e isolamento da lógica de criação utilizando o padrão Factory Method (FuncionarioFactory).
Modelagem de Cargos / Papéis	Utilização de um atributo enumerado dentro de uma única classe genérica Funcionario.	Substituição do Enum por herança e especialização de classes com o surgimento das subclasses Gerente e Estoquista.
Tipagem de Dados (Segurança e Máscaras)	Atributos estruturados com tipos primitivos inadequados (ex: CPF, Telefone e Data de Nascimento como int; Nome e Senha como char).	Correção estrita e tipagem forte nos atributos de Pessoa e Funcionario, passando a utilizar String, double e Date.
Regras e Validações de Domínio	Inexistência de rotinas de validação de dados estruturadas na entidade que gerencia as informações.	Adição do método especialista <code>validarCPF(cpf : String) : boolean</code> diretamente na classe de domínio Pessoa.is
Assinatura de Métodos de Cadastro	Métodos com longas listas de parâmetros primitivos na assinatura	Simplificação e encapsulamento das operações de ciclo de vida (ex: <code>cadastrarPessoa() : void</code>), favorecendo o uso de objetos de transferência.

2 Documento Arquitetural – Aplicação dos Padrões GoF

Os cinco padrões aplicados foram escolhidos de forma que nenhum coincidissem com os padrões apresentados por outros grupos durante os seminários. Foram selecionados os padrões Factory Method, Singleton, Facade, Strategy e Template Method, aplicados estrategicamente aos domínios de Controle de Acesso e Gerenciamento de Estoque de Produtos Perecíveis.

2.1 Padrão 1: Factory Method [Criacional]

Problema Identificado

O sistema precisava criar diferentes tipos de Produto (perceíveis, secos e refrigerados) e Lote com configurações distintas. Na versão original, os objetos eram instanciados diretamente com `new` no código cliente, espalhando a lógica de criação por todo o sistema. Isso tornava impossível variar o tipo de objeto criado sem modificar cada ponto de instância, violando o Princípio Aberto/Fechado (OCP).

Solução Proposta

Aplicação do padrão Factory Method. A interface `FuncionarioFactory` declara o método `criarFuncionario() : Funcionario`; As classes concretas `GerenteFactory` e `EstoquistaFactory` implementam essa interface, assumindo a responsabilidade de instanciar e configurar os respectivos objetos especializados (Gerente e Estoquista). O código cliente passa a operar exclusivamente através do contrato das fábricas.

Justificativa Técnica

O Factory Method delega a decisão de qual classe concreta instanciar para as subclasses especialistas de criação, desacoplando o fluxo de execução principal da lógica de construção. No contexto de gestão de usuários, isso isola o processo de parametrização e controle de privilégios de acesso associados a cada cargo.

Por que usar o padrão?

O sistema gerencia múltiplos papéis de usuários com responsabilidades distintas. O Factory Method centraliza as regras de construção, elimina condicionais complexas de validação no cliente e permite a adição de novos cargos futuramente sem a necessidade de alterar ou quebrar o código existente.

Impacto Arquitetural

A lógica de criação foi encapsulada nas fábricas concretas. Componentes externos ou chamadores dependem unicamente da abstração `FuncionarioFactory`, elevando a exten-

sibilidade da arquitetura do módulo de identidade e autenticação do sistema

Vantagens

- Centraliza e encapsula as regras de construção;
- Permite adicionar novos tipos de produto sem modificar código existente;
- Facilita testes unitários;
- Elimina duplicação de código de validação.

Possíveis Limitações

- Introduce uma quantidade maior de classes e interfaces no ecossistema do projeto;
- Exige maior maturidade do código consumidor para depender de abstrações.

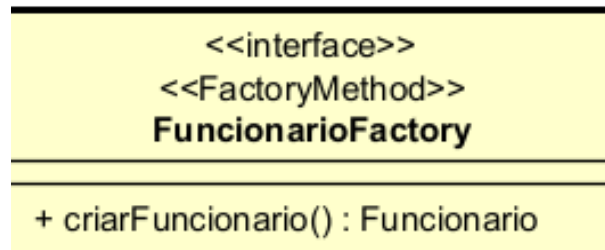


Figura 7: Diagrama:FuncionarioFactory

2.2 Padrão 2: Singleton [Criacional]

Problema Identificado

A classe que gerencia as operações físicas do inventário é o núcleo crítico do sistema. Na versão original (denominada **GerenciadorEstoque**), era possível instanciar múltiplos objetos deste gerenciador, permitindo que diferentes partes da aplicação guardassem estados de inventário distintos, o que gerava gravíssimas inconsistências de dados e duplicação de registros em memória.

Solução Proposta

Aplicação do padrão Singleton diretamente na classe **Estoque**. O construtor padrão da classe foi encapsulado com visibilidade privada (`-Estoque()`) e a referência para a sua única instância passou a ser controlada por um atributo estático interno (`-instancia : Estoque`). O acesso global unificado ao recurso foi disponibilizado exclusivamente pelo método estático público `+getInstance() : Estoque`.

Justificativa Técnica

Em sistemas logísticos e de estocagem em tempo real, as movimentações de lotes devem ser atômicas e centralizadas. A imposição de uma instância única e global assegura que todas as chamadas operem sobre a mesma coleção interna de dados físicos (`-lotes : List<Lote>`), mantendo a integridade do inventário.

Por que usar o padrão?

A classe **Estoque** é um recurso compartilhado vital. O Singleton assegura uma fonte única da verdade para o estado atual da aplicação, prevenindo concorrência desordenada e leitura de dados obsoletos de lotes armazenados.

Impacto Arquitetural

Todos os subsistemas da aplicação e a camada de interface passam a consumir o inventário obrigatoriamente via chamada estática `Estoque.getInstance()`, consolidando a persistência em memória e uniformizando o gerenciamento físico dos lotes.

Vantagens

- Garante consistência do estado do inventário;
- Fornece acesso global controlado;
- Permite inicialização sob demanda;
- Serve de barreira contra instanciações acidentais em outras camadas.

Possíveis Limitações

- Pode dificultar testes unitários;
- Introduz um acoplamento implícito que pode ocultar falhas de arquitetura orientada a componentes.

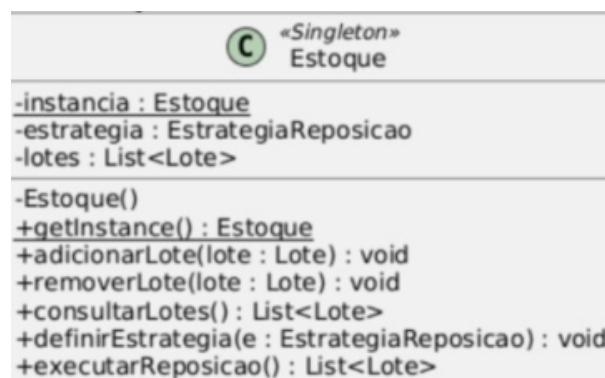


Figura 8: Diagrama:Singleton

2.3 Padrão 3: Facade [Estrutural]

Problema Identificado

O ecossistema de estoque é composto por um conjunto complexo de interações entre as entidades **Fornecedor**, **Produto** e **Lote**. Na modelagem original, os módulos consumidores externos necessitavam orquestrar e chamar de forma sequencial os métodos e ciclos de vida de cada uma dessas classes para concluir tarefas básicas (como cadastros cruzados), gerando alto acoplamento, código redundante e fragilidade a alterações internas.

Solução Proposta

Aplicação do padrão Facade com a introdução da classe **SistemaEstoqueFacade**. Esta classe assume o papel de fachada do subsistema, centralizando operações complexas em métodos simplificados de alto nível para o cliente, como `+cadastrarProduto()`, `+cadastrarFornecedor()`, `+registrarLote()`, e métodos de delegação como `+executarFEF0()` e `+gerarRelatorioPerdas()`.

Justificativa Técnica

O padrão Facade reduz de forma drástica a complexidade operacional da aplicação ao disponibilizar uma interface limpa, mascarando a colaboração interna de objetos do domínio. Isso protege a integridade dos fluxos do sistema e isola mudanças de refatoração internas da camada cliente..

Por que usar o padrão?

Processar o recebimento ou auditoria de itens perecíveis envolve disparar fluxos coordenados de validação. A fachada encapsula essa orquestração complexa de subsistemas em assinaturas únicas, facilitando significativamente a integração do sistema.

Impacto Arquitetural

A classe **SistemaEstoqueFacade** se estabelece como a fronteira oficial e o ponto central de comunicação entre agentes externos (interfaces e APIs) e as classes internas de negócio (**Fornecedor**, **Produto**, **Lote**, **Estoque**), elevando os níveis de coesão interna.

Vantagens

- Reduz o acoplamento entre os consumidores e as classes do domínio;
- Simplifica a utilização do sistema;
- Padroniza e garante a execução das regras de negócio organizadas na sequência correta;
- Facilita modificações internas no domínio sem quebrar sistemas dependentes.

Possíveis Limitações

- Há o risco de a fachada se transformar em um God Object (objeto onisciente), acumulando responsabilidades excessivas se não houver disciplina na distribuição das subcamadas.

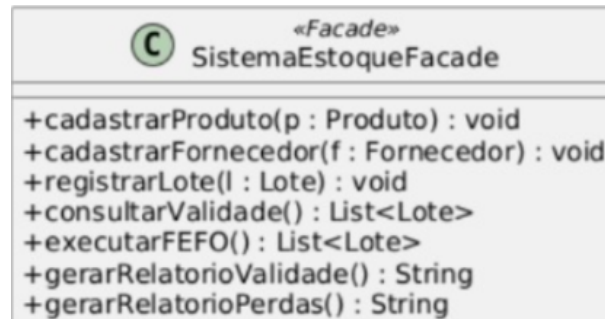


Figura 9: Diagrama:Facade

2.4 Padrão 4: Strategy [Comportamental]

Problema Identificado

O sistema exige suporte a diferentes lógicas e algoritmos logísticos de triagem e vazão de mercadorias perecíveis (como priorizar o vencimento ou a data de entrada). Na abordagem original, estes critérios de negócio estavam engessados em blocos condicionais encadeados no núcleo da classe que manipulava os dados, tornando a manutenção onerosa e ferindo frontalmente o princípio Open/Closed (OCP).

Solução Proposta

Aplicação do padrão Strategy. Foi implementada a interface abstrata **EstrategiaReposicao**, definindo o contrato polimórfico **+ordenarSaida(lotes : List<Lote>) : List<Lote>**. Os algoritmos foram encapsulados em suas próprias classes independentes: **FEFOStrategy** (priorização por data de vencimento) e **FIFOStrategy** (priorização por ordem de chegada). A classe **Estoque** armazena dinamicamente a estratégia atual via agregação pelo atributo **-estrategia : EstrategiaReposicao**.

Justificativa Técnica

O padrão substitui decisões lógicas estruturais por composição de objetos. O método **+executarReposicao()** da classe **Estoque** simplesmente transfere a chamada de processamento de ordenação para o objeto configurado através do método dinâmico **+definirEstrategia(e : EstrategiaReposicao)**.

Por que usar o padrão?

Diferentes categorias de perecíveis necessitam de tratamentos de vazão distintos sob regulamentações sanitárias específicas. O Strategy habilita o intercâmbio fluido de com-

portamentos de triagem logística em tempo de execução, permitindo escalar novas lógicas de despacho sem qualquer intervenção no código fonte do Estoque.

Impacto Arquitetural

O desacoplamento isolou os algoritmos complexos de ordenação logística da estrutura de dados que retém os lotes armazenados, viabilizando testes unitários focados e isolados por algoritmo de distribuição.

Vantagens

- Elimina condicionais complexas;
- Torna o código do gerenciador estável e focado na sua responsabilidade primária;
- Concede maleabilidade comportamental ao sistema em tempo de execução;
- Favorece a conformidade com regras de negócio específicas.

Possíveis Limitações

- Requer conhecimento das estratégias disponíveis;
- O cliente precisa estar ciente das implementações de estratégia disponíveis para configurá-las corretamente.

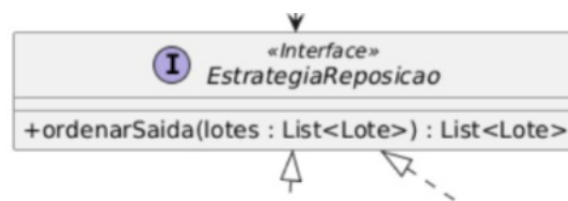


Figura 10: Diagrama: Strategy



Figura 11: Diagrama: Strategy2

2.5 Padrão 5: Template Method [Comportamental]

Problema Identificado

A emissão de documentos de controle gerencial (relatórios de perdas de insumos e relatórios de conformidade de validade) compartilhava o mesmo fluxo operacional sequencial: extrair as coleções de dados, aplicar filtragens/processamentos e realizar a formatação final da

string para exibição. Na versão original, essa rotina metodológica estava replicada nas classes, acarretando redundância de código e quebra na padronização dos documentos gerados.

Solução Proposta

Este método orquestra a chamada encadeada das rotinas protegidas e abstratas `#coletarDados()`, `#processarDados()` e `#formatar()`. As subclasses filhas concretas `RelatorioValidade` e `RelatorioPerdas` realizam a herança e estendem estritamente o comportamento variável das etapas, mantendo a sequência principal intocada.

Justificativa Técnica

O Template Method garante o determinismo estrutural do algoritmo principal por meio de herança clássica. Ele confere às subclasses o direito de redefinir passos individuais específicos de negócio sem a capacidade de alterar o esqueleto coordenador definido pela classe abstrata.

Por que usar o padrão?

Documentos de auditoria sanitária e controle interno exigem a preservação de padrões rígidos de layout e etapas corporativas. O padrão protege a estabilidade estrutural destes fluxos e otimiza a produtividade do time, centralizando o controle do esqueleto.

Impacto Arquitetural

Estabelecimento de uma inversão de controle clara na arquitetura de relatórios, em que a superclasse `Relatorio` determina de forma absoluta o momento e o contexto em que as lógicas customizadas das subclasses serão executadas.

Vantagens

- Elimina duplicação de código;
- Garante uma sequência padronizada de execução;
- Facilita a criação de novos relatórios;
- Reduz custos de manutenção ao isolar mudanças no fluxo geral em um único ponto abstrato.

Possíveis Limitações

- Acoplamento via herança: subclasses dependem da estrutura da classe abstrata, dificultando refatorações na hierarquia;

- Apresenta rigidez caso surja um novo modelo de relatório que demande passos completamente divergentes do template pré-estabelecido.

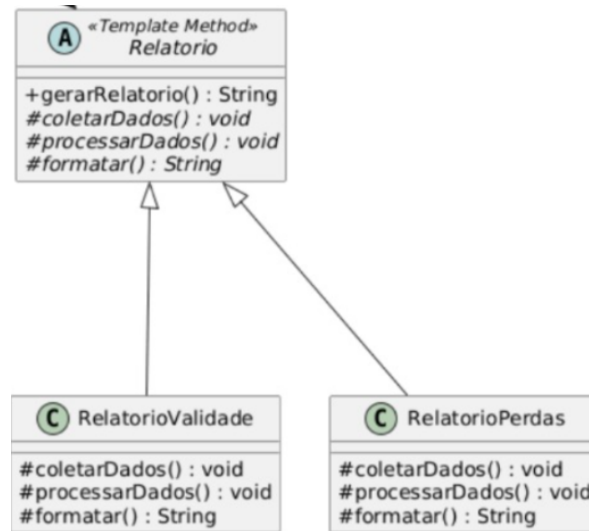


Figura 12: Diagrama:Method